LAB6: 컴퓨터와 FPGA 통신

실험 전 보고서

학과: 전자전기컴퓨터공학부

이름/학번: 정현재/2023440127

작성일: 2026/10/23

대상: PC 입출력/UART 통신 및 LED 제어/서보모터 제어/SRAM/DAC

검증 환경: VS Code VaporView, STM32 빌드 도구

보고서 및 실험의 범위

STM32 과제 3개, FPGA 실험 3개를 대상으로 설계와 사전 검증을 정리하였다. STM32는 PC 입력 문자의 문자·16진수 출력, UART를 통한 FPGA 데이터 전달 및 LED 제어 코드를 작성하고 빌드 결과를 확인하였다. FPGA는 서보모터 제어·SRAM·DAC 회로를 테스트벤치로 검증하고 파형 파일을 생성하였으며, Vivado에서 합성·구현·비트스트림 생성까지 수행하였다. 실제 보드 다운로드와 동작 측정은 수업에서 확인할 항목으로 구분하였다.

소스 식별

프로젝트: LAB6

기준 커밋: 188457537e1b79d45df16028014f5e050da389b

실행본: STM32 각 실험 폴더의 Core/Src/, Core/Inc/ 및 프로젝트 설정 파일, FPGA 각 실험 폴더의 src/, sim/, constraints/ 및 simulation.json

GitHub 주소: https://github.com/Jeong719/ECE2_Lab6.git

제출 태그: v1.1

1. 목적과 공통 검증 조건

실험 목적

본 실험의 목적은 STM32의 PC 입출력과 UART 통신 방법을 이해하고, FPGA를 이용하여 서보모터 제어, SRAM 읽기·쓰기 및 DAC 인터페이스를 구현하는 것이다.
STM32에서는 PC에서 받은 문자와 바이트 값을 출력하고, UART를 통해 FPGA에 문자와 LED 제어 명령을 전달한다. FPGA에서는 펄스 폭에 따른 서보모터의 위치 제어, 외부 SRAM의 데이터 저장과 읽기, 디지털 값에 따른 DAC의 아날로그 출력을 구현하고 시뮬레이션과 실제 보드에서 동작을 확인한다.

관련 이론

SRAM: SRAM은 전원이 공급되는 동안 데이터를 유지하는 휘발성 메모리이다.
SRAM 셀은 교차 연결된 인버터의 피드백을 이용하여 비트 값을 저장하므로, DRAM과 달리 주기적인 리프레시가 필요하지 않다[1]. 주소 신호로 저장 위치를 선택하고, 칩 선택(CS), 쓰기 허용(WE), 출력 허용(OE) 신호로 읽기·쓰기 동작을 제어한다.
데이터 버스는 양방향이므로 읽기 시 FPGA 출력을 고임피던스(Z) 상태로 설정하여 버스 충돌을 방지한다. 본 실험에서는 주소 0~15에 4비트 데이터를 저장하고, 읽은 값을 LED로 확인한다.

DAC: DAC는 디지털 값을 그에 대응하는 아날로그 전압이나 전류로 변환하는 회로이다. 8비트 DAC는 256개의 디지털 입력 값에 대응하는 출력 단계를 가지며, 같은 출력 범위에서 비트수가 클수록 더 세밀한 변환이 가능하다. 본 실험에서는 8비트 데이터를 쓰기 제어 신호와 함께 DAC에 전달하고, 값을 0~255로 반복 증가시켜 아날로그 출력 전압이 단계적으로 변하는 것을 확인한다.

검증 환경과 실행 방법

STM32는 네 프로젝트를 각각 빌드하여 컴파일 및 링크의 성공 여부를 확인한다. FPGA는 Icarus Verilog로 시뮬레이션하고 VCD 파일을 VaporView로 열어 분석한다. 두 FPGA 회로에는 클록이 없으며, TB는 입력을 1ns 간격으로 변경한다.

통과 기준과 파형 분석 기준

STM32의 경우 디버그 및 빌드 후 build finished with exit code 0가 출력되는 것, FPGA 실험의 경우 정상 및 복구 실행은 테스트벤치의 모든 검사를 통과하여 PASS 로그가 공통적으로 출력되는 것을 통과 기준으로 한다. 실행 로그의 검사 수와 종료 시각을 기록하고 전체 파형과 각 실험마다 주요 기능이 담긴 구간의 확대 파형을 통해 조건에 따라 올바르게 출력됐는지 분석한다.

시간 단위와 검증 범위

파형의 시간 단위는 ns를 기준으로 해석하며, ps로 표시된 경우 1000ps를 1ns로 환산한다. TB의 10ns 클록은 기능 검증을 위한 조건이다.

2. STM32 과제 5: PC와 MCU 출력

설계 파일: 03_task5_pc_hex의

Core/Src/ece2_474_routine.c

설계와 예상 결과

본 과제는 PC에서 입력한 문자와 해당 문자의 16진수 값을 serial 모니터에 출력한다. 예를 들어 A를 입력하면 0x41이 serial 모니터에 표시된다. 핵심코드는 다음과 같다.

int ch = getchar();
uint8_t data = (uint8_t)ch;
printf("RX: %c (0x%02X)₩r₩n", data, (unsigned int)data);

해당 코드를 통해 PC에서 문자 한 개를 받고 그 받은 값을 문자와 두 자리 16진수로 출력한다.

사전 검증과 파형 해석

```
[driver] Build completed: 00:00:03.813
[build] Build finished with exit code 0
```

사전 검증 결과 오류 없이 빌드가 완료된 것을 확인할 수 있다.

보드에서 확인할 것

PC에 입력한 문자에 대응하는 아스키 코드가 성공적으로 출력되는지 확인한다.

3. STM32 과제 6: STM32에서 FPGA로 문자 전달

설계 파일: 04_task6_fpga_echo의
Core/Src/ece2_474_routine.c

설계와 예상 결과

본 과제는 PC에서 받은 문자를 STM32가 UART3로 FPGA에 전달하고, FPGA가 PC로 출력한 문자와 원래 입력을 비교한다.

HAL_StatusTypeDef result = HAL_UART_Transmit(&huart3, &data, 1U, 100U);
printf("UART3 TX status: %d₩r₩n", (int)result);

해당 코드들을 통해 UART3의 PC10으로 한 바이트를 전송하며 최대 100ms 기다린다. Result는 STM32의 송신결과다.

사전 검증과 파형 해석

```
[driver] Build completed: 00:00:03.535
[build] Build finished with exit code 0
```

사전 검증 결과 오류 없이 빌드가 완료된 것을 확인할 수 있다.

보드에서 확인할 것

PC에서 입력한 문자가 정확히 serial monitor에 잘 출력되는지 확인한다. 이 때 출력은 아스키 코드가 아닌 입력한 문자 그 자체인 것을 유의한다.

4. STM32 과제 7: UART 명령으로 FPGA LED 제어

설계 파일:
05_task7_fpga_led의
Core/Src/ece2_474_routine.c

설계와 예상 결과

본 과제는 UART를 활용하며, PC에서 0을 입력하면 FPGA의 LED를 끄고, 1을 입력하면 점등하고, 2는 LED 상태를 반전시키며, 다른 문자는 상태를 유지한다.
핵심코드는 다음과 같다.

HAL_UART_Transmit(&huart3, &data, 1U, 100U);

이 코드를 통해 STM32는 입력받은 명령

문자를 FPGA로 전달한다. 아래 코드를 통해 FPGA는 명령을 해석하고 LED 상태를 변경한다.

```
case (rx_data)
    8'h30: last_data <= 0;
// '0': 끄기
    8'h31: last_data <= 1;
// '1': 켜기
    8'h32: last_data <= last_data ^
8'h01;   // '2': LED[0] 반전
    default: last_data <= last_data;
// 상태 유지
endcase
```

사전 검증과 파형 해석

```
[driver] Build completed: 00:00:03.526
[build] Build finished with exit code 0
```

사전 검증 결과 오류 없이 빌드가 완료된 것을 확인할 수 있다.

보드에서 확인할 것

0을 입력했을 때 LED가 소등되는지, 1을 입력했을 때 점등되는지, 2를 입력했을 때는 이전 상태를 반전시키는지, 다른 문자에 대해서는 변화가 없는지 확인한다.

5. FPGA 실험 1: 서보모터 제어

설계 파일:

servo_ctrl.v, servo_position.v, servo_pwm.v

설계와 예상 결과

본 회로는 좌우 버튼으로 서보모터의 위치를 조절하는 회로다. 20ms 주기의 제어 신호에서 HIGH 구간을 1~2ms로 바꾸며, 초기 펄스 폭은 중앙 위치에 해당하는 1.5ms다. 버튼을 한 번 누르면 0.1ms씩 변경된다.

핵심코드는 다음과 같다.

```
if(rst_p) pulse_us <= 1500;
else if(left_pulse && !right_pulse &&
pulse_us>1000)
    pulse_us <= pulse_us-100;
else if(right_pulse && !left_pulse &&
pulse_us<2000)
    pulse_us <= pulse_us+100;
```

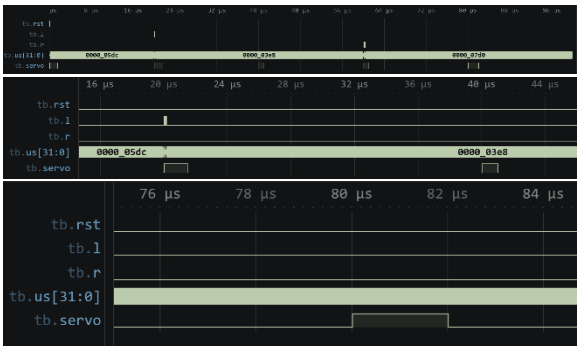
해당 코드를 통해 왼쪽 버튼은 펄스 폭을 줄이고 오른쪽 버튼은 늘리는 역할을 수행할 수 있다.

사전 검증과 파형 해석



사전 검증 결과 총 100036ns가 소요되어 정상적으로 pass 문구가 뜨는 것을 확인할 수 있다.

아래는 서보모터 제어 회로의 전체적인 파형과 주목해야 할 부분에 대한 사진이다.



두 번째 사진의 18~43μs 구간에서 약 20 μs 간격으로 나오는 펄스와 HIGH 폭이 1.5μs에서 1μs로 변화하는 모습을 확인할 수 있다.

세 번째 사진에서 us=2000, width_cycle=200이고 HIGH 폭이 2μs인 것을 확인할 수 있다.

보드에서 확인할 것

좌우 버튼을 눌렀을 때 서보모터의 위치가 정상적으로 알맞은 방향으로 조절되는지 확인한다. 두 버튼을 동시에 눌렀을 때 서보모터가 움직이지 않는지 확인한다.

6. FPGA 실험 2: SRAM

설계 파일:

Sram.v, sram_bus_control.v

설계와 예상 결과

본 회로는 스위치로 주소와 데이터를 선택한 뒤, 쓰기 버튼으로 외부 SRAM에 저장하고 읽기 버튼으로 저장값을 확인하는 회로다. 주소 0~15와 4비트 데이터를 사용하며, 읽은 값은 LED 하위 4개에 표시한다.

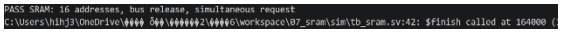
핵심코드는 다음과 같다.

```
assign drive_data = ctrl_w && !ctrl_r;
assign read_data  = ctrl_r && !ctrl_w;

assign ram_address = {14'b0,address};
assign ram_data = drive_data ? {12'b0,data_in} : 16'bz;
assign led_out = read_data ? {4'b0,ram_data[3:0]} : 8'b0;
```

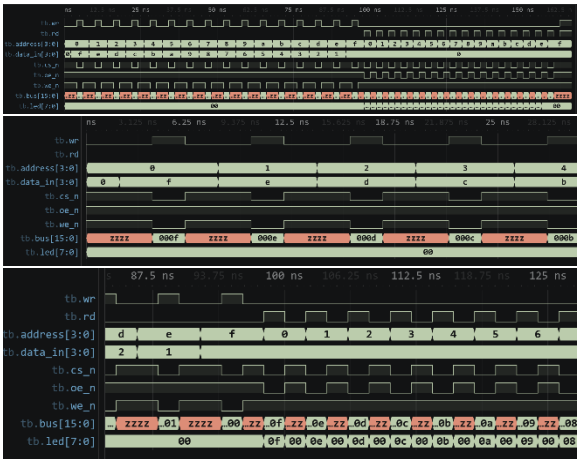
해당 코드에서 drvie_data는 쓰기만 요청했을 때, read_data는 읽기만 요청했을 때 활성화된다. 쓰기 중에는 FPGA가 ram_data에 데이터를 내보낸다.

사전 검증과 파형 해석



사전 검증 결과 총 16개의 주소값, 164ns가 소요되어 정상적으로 pass 문구가 뜨는 것을 확인할 수 있다.

아래는 SRAM 회로의 전체적인 파형과 주목해야 할 부분에 대한 사진이다.



두 번째 사진에서 2~26ns에서 wr=1이 되어 쓰기 신호가 활성화되고 주소 0, 1, 2, 3에 데이터 F, E, D, C를 쓰는 모습을 확인할 수 있다.

세 번째 사진에서 98~122ns에서 같은 주소를 읽으면 bus와 led에 저장했던 F, E, D, C가 나오는 모습을 확인할 수 있다.

보드에서 확인할 것

쓰기모드로 SRAM에 특정 주소에 데이터를 저장하고 읽기모드로 해당 데이터를 불러왔을 때 저장한 값과 일치하는지 확인한다.

7. FPGA 실험 3: DAC

설계 파일:

Dac.v, dac_parallel8.v, ramp8.v

설계와 예상 결과

본 회로는 8비트 디지털 값을 일정 간격으로 증가시켜 외부 DAC에 전달한다. DAC는 이 값을 아날로그 전압으로 변환하므로,

출력 전압이 단계적으로 증가하다가 다시 낮아지는 파형을 만든다.

핵심코드는 다음과 같다.

assign update = (count == UPDATE_CYCLES-1);

```
always @(posedge clk_50mhz) begin
    if(rst_p) begin count<=0; value<=0;
end
    else if(update) begin count<=0;
value<=value+1'b1; end
    else count<=count+1;
end
```

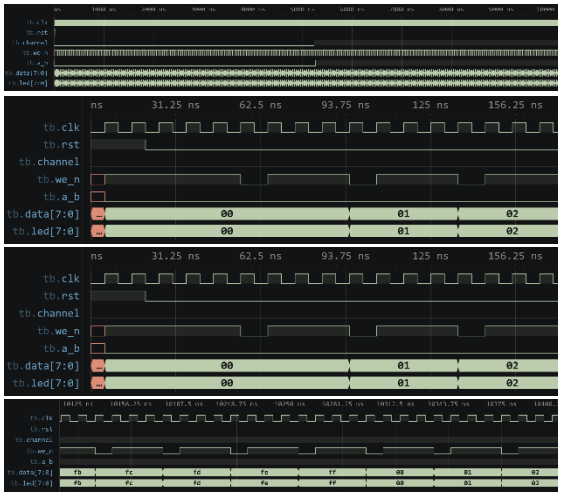
해당 코드를 통해 카운터가 설정값에 도달하면 update가 활성화되고 value가 1 증가한다. Value는 8비트이므로 255 다음에는 자동으로 0이 된다.

사전 검증과 파형 해석



사전 검증 결과 총 260개의 쓰기와 10436ns가 소요되어 정상적으로 pass 문구가 뜨는 것을 확인할 수 있다.

아래는 DAC 회로의 전체적인 파형과 주목해야 할 부분에 대한 사진이다.



두 번째 사진에서 data가 00->01->02로 증가하고, we_n이 주기적으로 LOW가 되는 모습을 확인할 수 있다.

세 번째 사진에서 channel이 바뀐 뒤 다음 쓰기에서 a_b에 반영되는 모습을 확인할 수 있다.

네 번째 사진에서 data가 FE->FF->00->01로 255 이후 다시 0으로 돌아가는 모습을 확인할 수 있다.

보드에서 확인할 것

출력 전압이 정상적으로 높아졌다가 낮아지는 파형을 형성하는지 확인한다.

8. 실험 수행 계획

1 단계(프로젝트 준비 및 코딩)

VS Code 에서 STM32 의 제공 예제와 과제 프로젝트를 준비하고, PC 입출력과 UART 통신 코드를 작성한다. FPGA 는 서보모터 제어, SRAM 읽기·쓰기, DAC 의 RTL 및 테스트벤치 파일을 준비한다.

2 단계(빌드 및 기능 검증)

STM32 는 각 프로젝트를 빌드하여 컴파일 오류가 없는지 확인한다. FPGA 는 테스트벤치를 실행하여 입력에 따른 출력값을 확인하고, 예상 결과와 비교하여 기능을 검증한다.

3 단계(결과 분석)

STM32 는 코드를 분석하여 PC 에서 받은 문자와 16 진수 값의 출력, UART 를 통한 FPGA 데이터 전달 및 LED 제어 동작을 예상한다. FPGA 는 시뮬레이션 파형에서 서보모터의 제어 펄스 주기와 폭, SRAM 의

쓰기·읽기 동작, DAC 의 데이터 변화와 쓰기 제어 신호가 올바른지 확인한다.

4 단계(보드 구현 및 동작 확인)

STM32 는 프로그램을 MCU 보드에 다운로드하여 제공 예제 2 개와 과제 3 개의 동작을 확인한다. PC 의 시리얼 모니터에서 문자와 16 진수 출력을 확인하고, FPGA 와 UART 로 연결하여 문자 전달 및 명령에 따른 LED 상태 변화를 확인한다. FPGA 는 Vivado 에서 핀 제약을 적용하고 합성, 구현 및 비트스트림 생성을 수행한 뒤 보드에 다운로드하여 서보모터의 위치 변화와 SRAM 저장값의 LED 표시를 확인하고, 오실로스코프로 DAC 의 아날로그 출력 파형을 측정한다.

9. 참고 문헌

[1] L. Harris and D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2022, Sec. 5.5.3, "Static Random Access Memory (SRAM)." ISBN: 978-0-12-820064-3.